

MASTER.C

```

/*****
Code(Master) for SPI Communication between two ATMEGA2560 Board
*****/
#include <ioavr.h>
#include <intrinsics.h>

#define LED1 PORTL6 //Connected to PORTL
#define LED2 PORTL7 //Connected to PORTL
#define LED3 PORTJ4 //Connected to PORTJ
#define LED4 PORTJ7 //Connected to PORTJ

#define SW1 PINJ0 //All Switches are connected to PORTJ
#define SW2 PINJ1
#define SW3 PINJ2
#define SW4 PINJ3

#define MOSI PB2
#define SCK PB1
#define SS PB0

#define SETBIT(ADDRESS, BIT) (ADDRESS |= (1 << BIT))
#define CLEARBIT(ADDRESS, BIT) (ADDRESS &= ~(1 << BIT))
#define CHECKBIT(ADDRESS, BIT) ((ADDRESS & (1 << BIT)))

//SPI Subroutines
//Function to Initialize SPI as Master
void SPI_MasterInit(void)
{
    //Set MOSI, SCK and SS output, all others
    ers input
    DDOR = (1<<MOSI)|(1<<SCK)|(1<<SS);
    //Enable SPI, Master, set clock rate
    fosc/16 - 115.2 kHz SPI with 3.686 Mhz Osc
    SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR0);
    CLEARBIT(PORTS, SS); //Select the slave device
}

//Function to send byte through SPI Interface, ignoring the returning byte
void SPI_SendByte(unsigned char byte)
{
    SPDR = byte;
    while(!(SPSR & (1<<SPIF))); // Wait for transmission complete
}

//Assuming Internal Clock of 3.686 Mhz
C_task void main(void)
{
    //Set all LEDs pins as Output
    SETBIT(DDRL, LED1);
    SETBIT(DDRL, LED2);
    SETBIT(DDRJ, LED3);
    SETBIT(DDRJ, LED4);

    //Enable Pull-ups on PORTJ Switches
    PORTJ |= (1<<PJ0)|(1<<PJ1)|(1<<PJ2)|(1<<PJ3);

    SPI_MasterInit();
    while(1)
    {
        if (CHECKBIT(PINJ, SW1))
        {
            SPI_SendByte('A');
            SETBIT(PORTL, LED1);
        }
        else
        {
            CLEARBIT(PORTL, LED1);
        }
        if (CHECKBIT(PINJ, SW2))
        {
            SPI_SendByte('B');
            SETBIT(PORTL, LED2);
        }
        else
        {
            CLEARBIT(PORTL, LED2);
        }
        if (CHECKBIT(PINJ, SW3))
        {
            SPI_SendByte('C');
            SETBIT(PORTJ, LED3);
        }
        else
        {
            CLEARBIT(PORTJ, LED3);
        }
        if (CHECKBIT(PINJ, SW4))
        {
            SPI_SendByte('D');
            SETBIT(PORTJ, LED4);
        }
        else
        {
            CLEARBIT(PORTJ, LED4);
        }
    }
}

```

SLAVE.C

```

/*****
Code(Slave) for SPI Communication between two ATMEGA2560 Board
*****/
#include <ioavr.h>
#include <intrinsics.h>

#define LED1 PORTL6 //Connected to PORTL
#define LED2 PORTL7 //Connected to PORTL
#define LED3 PORTJ4 //Connected to PORTJ
#define LED4 PORTJ7 //Connected to PORTJ

#define MISO PB3

#define SETBIT(ADDRESS, BIT) (ADDRESS |= (1 << BIT))
#define CLEARBIT(ADDRESS, BIT) (ADDRESS &= ~(1 << BIT))

//Function to Initialize SPI as Slave
void SPI_SlaveInit(void)
{
    //Set MISO output, all others input */
    DDOR = (1<<MISO);

    /* Enable SPI */
    SPCR = (1<<SPE);
}

//Function to receive byte from the master
SPI device
unsigned char SPI_SlaveReceive(void)
{
    //Wait for reception complete */
    while(!(SPSR & (1<<SPIF)));
    /* Return data register */
    return SPDR;
}

//Assuming External Clock of 16 Mhz
C_task void main(void)
{
    //Set all LEDs pins as Output
    SETBIT(DDRL, LED1);
    SETBIT(DDRL, LED2);
    SETBIT(DDRJ, LED3);
    SETBIT(DDRJ, LED4);

    unsigned char k;
    SPI_SlaveInit();
    while(1)
    {
        k = SPI_SlaveReceive();
        if (k == 'A')
        {
            SETBIT(PORTL, LED1);
        }
        else
        {
            CLEARBIT(PORTL, LED1);
        }
        if (k == 'B')
        {
            SETBIT(PORTL, LED2);
        }
        else
        {
            CLEARBIT(PORTL, LED2);
        }
        if (k == 'C')
        {
            SETBIT(PORTJ, LED3);
        }
        else
        {
            CLEARBIT(PORTJ, LED3);
        }
        if (k == 'D')
        {
            SETBIT(PORTJ, LED4);
        }
        else
        {
            CLEARBIT(PORTJ, LED4);
        }
    }
}

```

USART.C

```

/*
Serial Communication Check Source for ATMEGA2560 Board
*/
#include <ioavr.h>

#define SETBIT(ADDRESS, BIT) (ADDRESS |= (1 << BIT))
#define CLEARBIT(ADDRESS, BIT) (ADDRESS &= ~(1 << BIT))
#define CHECKBIT(ADDRESS, BIT) (ADDRESS & (1 << BIT))

#define LED1 PORTL6 //LED Connected to PORTL

/* Prototypes */
void USART_Init (unsigned int baud);
void USART_Transmit (unsigned char data);
unsigned char USART_Receive (void);

C_task void main (void)
{
    /* Set the baudrate to 115.2 kbps using a 16.0 Mhz crystal
    Fuse bits set to handle CPU with external Crystal Oscillator */
    unsigned char a = 0;
    USART_Init (8);
    SETBIT(DDRL, PL6); //Set LED pin as output
    while (1)
    {
        a = USART_Receive();
        if (a == '0') CLEARBIT(PORTL, LED1);
        if (a == '1') SETBIT(PORTL, LED1);
        USART_Transmit(a + 1);
    }

    //Initialize USART0
    void USART0_Init(unsigned int baud)
    {
        /* Set baud rate */
        UBRR0 = (unsigned char) (baud >> 8);
        UBRR0F = (unsigned char) (baud);
        /* Enable receiver and transmitter */
        UCSRB = (1<<RXEN0)|(1<<TXEN0);
        /* Set frame format: Asynchronous 8 data, 1 stop bit */
        UCSRC = (1<<UCS201)|(1<<UCS200);
    }

    //Read and Write functions for USART0
    void USART0_Transmit (unsigned char data)
    {
        /* Wait for empty transmit buffer */
        while (!CHECKBIT(UCSR0A, UDRE0));
        /* Put data into buffer, transmits the data */
        UDR0 = data;
    }

    unsigned char USART0_Receive (void)
    {
        /* Wait for data to be received */
        while (!CHECKBIT(UCSR0A, RXC0));
        /* Get and return received data from buffer */
        return UDR0;
    }
}

```